Chapter 5. 외부 도구 연동 (Tool Calling 실전)

> ⏱️ **배정 시간 60분** — Workflow as Tool 개념 10분 · 실습 5-1 15분 · 실습 5-2 10분 · 실습 5-3 10분 · 실습 5-4 15분

📋 학습 목표

이 챕터를 마치면 다음을 할 수 있습니다.

- [ ] 서브 워크플로우를 만들고 `Call n8n Workflow Tool`로 에이전트에 연결할 수 있다
- [ ] Workflow Input Schema를 정의해 에이전트가 채울 입력값을 지정할 수 있다
- [ ] 에이전트가 대화 중 필요할 때 스스로 외부 API(날씨·환율·공휴일)를 조회하게 만들 수 있다
- [ ] 여러 아이템으로 돌아온 API 결과를 Aggregate 노드로 하나의 문장으로 합칠 수 있다
- [ ] Tool의 Description이 호출 판단에 얼마나 큰 영향을 주는지 실험으로 확인할 수 있다
- [ ] 한 질문에 여러 Tool을 연쇄 호출하는 에이전트를 완성할 수 있다

📖 이 장의 용어

용어	영문 · 약어 풀이	설명
Workflow as Tool	도구로서의 워크플로우	워크플로우 자체를 Agent의 Tool로 등록하는 방식. n8n으로 만든 모든 작업을 Agent가 호출 가능
서브 워크플로우	Sub-workflow, 하위 워크플로우	다른 워크플로우에서 호출되어 하나의 작업을 맡는 워크플로우. 호출하는 쪽이 메인
Execute Sub-workflow Trigger	When Executed by Another Workflow	서브 워크플로우의 시작 노드. 다른 노드가 호출할 때 실행되며, 여기에 입력 스키마를 정의
Input Schema	입력 스키마, 입력 정의	어떤 이름·타입의 값을 받을지 미리 정해 둔 틀. Agent는 이 정의와 설명을 보고 값을 채움. 1일차 함수의 매개변수에 해당
Call n8n Workflow Tool	—	서브 워크플로우를 Agent의 Tool로 연결하는 노드. Description이 Agent의 호출 판단 기준

용어	영문 · 약어 풀이	설명
AI 버튼	$fromAI	입력 필드의 값을 사람이 정하지 않고 Agent가 대화 맥락으로 채우게 하는 설정
Aggregate	집계, 모으기	여러 아이템을 하나의 배열 필드로 합쳐 아이템 1개로 만드는 노드
배열	Array	값 여러 개를 순서대로 담은 목록. `.map()`은 각 항목을 변환, `.join()`은 하나의 문자열로 연결
대괄호 접근	Bracket notation	`rates[변수]`처럼 필드 이름이 실행 시점에 정해질 때 쓰는 문법. `rates.KRW`는 이름이 고정일 때
ISO 국가 코드 / 통화 코드	ISO 3166 / ISO 4217	국제 표준 약어. 국가는 2글자(KR, JP, US), 통화는 3글자(KRW, JPY, USD)
Open-Meteo / Frankfurter / Nager.Date	—	각각 날씨 · 환율 · 공휴일을 제공하는 무료 공개 API. 가입·API Key 불필요
ECB	European Central Bank, 유럽중앙은행	Frankfurter가 제공하는 환율의 출처. 영업일 하루 한 번 고시
연쇄 호출	Chained tool calls	한 질문에 Agent가 여러 Tool을 차례로 호출하고 결과를 합쳐 답하는 동작

5.1 Workflow as Tool

5.1.1 개념 — 워크플로우를 도구로 만든다

Chapter 4에서는 Calculator, Code Tool처럼 n8n이 기본 제공하는 도구를 썼습니다. 이번에는 한 단계 더 나아가 **내가 만든 워크플로우 자체를 하나의 Tool로 등록**합니다.

이게 왜 중요하냐면, n8n으로 만들 수 있는 거의 모든 작업(API 호출, 데이터 기록, 이메일 발송, 사내 시스템 연동)을 에이전트가 스스로 실행할 수 있게 되기 때문입니다. Tool의 종류에 제한이 사라지는 셈입니다.

> 🔗 **1일차 연결**: 1일차 Chapter 4에서 `get_current_weather(city)` 함수를 만들어 Tool로 등록했습니다. 오늘은 같은 일을 하는 **워크플로우**를 만들어 Tool로 등록합니다. 함수의 매개변수가 Input Schema로, Docstring이 Description으로, `return` 값이 마지막 노드의 출력으로 바뀔 뿐 구조는 같습니다.

구조

```
[메인 워크플로우]                    [서브 워크플로우 = Tool]
Chat Trigger → AI Agent  ──Tool──▶  Execute Sub-workflow Trigger
                                              ↓
                                     (실제 작업 노드들)
                                              ↓
                                     마지막 노드의 출력 = Tool의 결과
```

서브 워크플로우는 일반 워크플로우와 똑같이 만들되, 첫 노드로 `Execute Sub-workflow Trigger`를 씁니다. 캔버스에는 `When Executed by Another Workflow`로 표시됩니다. 이 트리거에서 **Workflow Input Schema**(입력 스키마: 어떤 이름과 타입의 값을 받을지 정해 둔 틀)를 정의하면, 에이전트가 이 Tool을 호출할 때 어떤 값을 채워야 하는지가 정해집니다. 그리고 **마지막 노드의 출력**이 그대로 에이전트에게 돌아갑니다.

설계 원칙

- 서브 워크플로우 하나는 **하나의 명확한 작업**만 담당하게 만듭니다. "날씨 조회"와 "환율 조회"는 따로 만듭니다.
- Input Schema의 필드명과 설명은 **에이전트가 읽고 이해할 수 있게** 씁니다. 에이전트는 이 설명을 보고 무엇을 채울지 결정합니다.
- 마지막 노드의 출력은 **에이전트가 읽기 쉬운 한 문장**으로 정리합니다. 긴 JSON을 그대로 돌려주면 답변 품질이 떨어집니다.

5.1.2 Tool 등록 방법

1. 메인 워크플로우의 AI Agent 노드에서 `Tool` 커넥터 `+` 클릭
2. `Call n8n Workflow Tool` 검색 후 추가
3. `Description` 작성 — 에이전트가 언제 이 Tool을 호출할지 판단하는 근거입니다
4. `Source for Workflow`를 `Database`로 설정하고, 미리 만들어둔 서브 워크플로우 선택
5. `Workflow Inputs` 영역의 **Refresh 버튼**을 눌러 Input Schema 필드 불러오기
6. 각 입력 필드 옆의 **AI 버튼**을 켜기 → 대화 내용에 따라 모델이 알아서 값을 채웁니다

> 💡 AI 버튼을 끄면 값을 사람이 직접 고정하거나 Expression으로 지정할 수 있습니다. 대화 내용에 따라 값이 달라져야 하는 경우(이 챕터의 실습이 그렇습니다)에는 켜두는 것이 일반적입니다.

✅ 체크포인트 질문

- 서브 워크플로우를 Tool로 쓸 때 에이전트에게 반환되는 값은? → **서브 워크플로우 마지막 노드의 출력**

- Description을 구체적으로 써야 하는 이유는? → **에이전트가 이 설명만 보고 호출 시점을 판단하기 때문**
- 1일차 Python Tool의 Docstring은 n8n에서 무엇에 해당하나요? → **Call n8n Workflow Tool의 Description + Input Schema의 필드 설명**

5.2 실습 5-1. 날씨 조회 Tool 만들기

> 🎯 **목표**: 날씨를 조회하는 서브 워크플로우를 만들어 단독으로 검증한다.
> ⏱️ **소요 시간 15분** · Chapter 1에서 쓴 Open-Meteo API 재사용 (인증 불필요)

Chapter 1에서는 우리가 좌표를 직접 넣었지만, 이번에는 **에이전트가 대화에서 지역을 파악해 좌표를 채워 넣습니다.** 같은 API인데 사용 주체가 사람에서 AI로 바뀌는 것입니다.

5.2.1 서브 워크플로우 만들기

1. 새 워크플로우 생성, 이름을 `Tool_날씨조회`로 설정
2. 첫 노드로 `Execute Sub-workflow Trigger` 추가 (검색 시 `When Executed by Another Workflow`)
3. `Input Data Mode`를 `Define using fields below`로 설정하고, 아래 세 필드를 추가

필드명	타입	Description
`latitude`	Number	조회할 지역의 위도. 예: 서울은 37.5665, 도쿄는 35.6762
`longitude`	Number	조회할 지역의 경도. 예: 서울은 126.9780, 도쿄는 139.6503
`location_name`	String	조회할 지역의 이름. 예: 서울, 도쿄

> 💡 **Description 칸을 비워두지 마세요.** 이 설명이 곧 에이전트에게 주는 지시입니다. "위도"라고만 쓰는 것과 "조회할 지역의 위도. 예: 서울은 37.5665"라고 쓰는 것은 결과가 확연히 다릅니다. 예시를 함께 넣으면 정확도가 크게 올라갑니다.

4. 뒤에 `HTTP Request` 노드 추가, Method는 `GET`으로 설정
5. URL 입력창을 fx 모드로 전환한 뒤 아래 Expression 입력

```
{{ 'https://api.open-meteo.com/v1/forecast?latitude=' + $json.latitude + '&longitude='
+ $json.longitude + '&current_weather=true' }}
```

6. 뒤에 `Edit Fields (Set)` 노드를 추가해 에이전트가 읽기 좋은 형태로 정리

Field Name	Value (Expression 모드)
`result`	`{{ $('When Executed by Another Workflow').item.json.location_name + '의 현재 기온은 ' + $json.current_weather.temperature + '도, 바람은 시속 ' + $json.current_weather.windspeed + 'km입니다.' }}`

> 💡 여기서 `$('When Executed by Another Workflow')`를 쓴 이유는, 지역 이름이 바로 앞 노드(HTTP Request)가 아니라 **맨 처음 트리거 노드**에 들어 있기 때문입니다. Chapter 1에서 배운 "다른 노드의 값 가져오기" 문법이 실제로 필요해지는 순간입니다.

7. 워크플로우 **저장** (저장하지 않으면 다음 단계에서 목록에 나타나지 않습니다)

5.2.2 단독 테스트

Tool로 연결하기 전에 서브 워크플로우 자체가 동작하는지 먼저 확인합니다.

서브 워크플로우의 트리거는 직접 실행하면 입력 필드가 비어 있는(`null`) 아이템을 내보냅니다. 테스트 값은 트리거의 출력을 **고정(Pin)**하는 방식으로 넣습니다.

1. 트리거 노드를 더블클릭해 열고 ▶(Execute step)으로 한 번 실행
2. 우측 OUTPUT 패널의 **연필 아이콘(Edit Output)** 클릭 → 아래 JSON을 입력하고 **Save** (출력이 고정되어 노드에 핀 표시가 나타납니다)

```
[{ "latitude": 35.6762, "longitude": 139.6503, "location_name": "도쿄" }]
```

3. 캠버스 하단의 **Execute workflow** 버튼으로 전체 실행 → 마지막 노드의 `result` 필드에 완성된 문장이 나오는지 확인
4. 확인이 끝나면 OUTPUT 패널의 **Unpin**을 눌러 고정을 해제하고 저장

> ⚠️ 서브 워크플로우는 반드시 단독으로 먼저 테스트하세요. 에이전트에 붙인 뒤에 오류가 나면 "에이전트가 호출을 안 한 건지, 호출은 했는데 워크플로우가 실패한 건지" 구분하기 어려워집니다.

5.3 실습 5-2. 환율 조회 Tool 만들기

> 🎯 **목표**: 통화 두 개를 입력받아 환율을 돌려주는 Tool을 만든다.
> ⏱️ **소요 시간 10분** · Chapter 1에서 쓴 Frankfurter API 재사용

구조가 실습 5-1과 같으므로 복제해서 고치면 빠릅니다.

1. `Tool_날씨조회`를 `···` → **Duplicate**로 복제하고 이름을 `Tool_환율조회`로 변경
2. 트리거 노드의 Input Schema를 아래 두 필드로 교체 (기존 세 필드는 삭제)

필드명	타입	Description
`base_currency`	String	기준 통화의 3자리 코드. 예: USD, JPY, EUR
`target_currency`	String	환산할 통화의 3자리 코드. 보통 KRW

3. HTTP Request 노드의 URL을 아래로 교체

```
{{ 'https://api.frankfurter.dev/v1/latest?base=' + $json.base_currency + '&symbols=' + $json.target_currency }}
```

4. Edit Fields 노드의 `result` 값을 아래로 교체

```
{{ '1 ' + $('When Executed by Another Workflow').item.json.base_currency + ' = ' + $json.rates[$('When Executed by Another Workflow').item.json.target_currency] + ' ' + $('When Executed by Another Workflow').item.json.target_currency + ' (' + $json.date + ' 유럽중앙은행 고시)' }}
```

> 💡 `$json.rates.KRW`가 아니라 `$json.rates[...]`로 쓴 이유는, 통화 코드가 고정이 아니라 **입력값에 따라 달라지기** 때문입니다. 대괄호 안에 변수를 넣으면 그 값의 이름을 가진 필드를 꺼냅니다.

5. 저장 후 5.2.2와 같은 방법으로 단독 테스트 (고정 데이터: `[{ "base_currency": "JPY", "target_currency": "KRW" }]`) → `1 JPY = 9.1 KRW (...)` 형태의 문장이 나오면 성공

5.4 실습 5-3. 공휴일 조회 Tool 만들기

> 🎯 **목표**: 국가 코드를 입력받아 다가오는 공휴일 목록을 한 문장으로 돌려주는 Tool을 만든다.
> ⏱️ **소요 시간 10분** · Nager.Date API 사용 (가입·API Key 불필요)

이 Tool에는 새로운 상황이 하나 있습니다. 공휴일 API는 결과를 **여러 개의 아이템**으로 돌려줍니다. Chapter 1에서 배운 "아이템 단위 흐름"을 실제로 다루게 됩니다.

5.4.1 API 확인

브라우저 주소창에 아래를 입력해 응답을 먼저 봅니다.

```
https://date.nager.at/api/v3/NextPublicHolidays/JP
```

오늘부터 1년 안에 있는 일본 공휴일이 배열로 돌아옵니다. 각 항목에 `date`(날짜), `localName`(현지 이름), `name`(영문 이름)이 있습니다. 한국은 `KR`, 미국은 `US`, 베트남은 `VN` 입니다.

> ⚠️ **강사 확인 사항**: Nager.Date 공식 저장소의 지원 일정표에 v3 API(이 실습에서 쓰는 주소)는 2027년 1월 31일 종료 예정으로 표기되어 있습니다. 그 이후 수업에서는 후속 버전(v4)의 주소와 응답 필드를 확인해 URL과 `localName` 참조를 수정해야 합니다. 커뮤니티 v4에서는 `localName` 필드가 제공되지 않는다는 보고가 있습니다.

5.4.2 서브 워크플로우 만들기

1. `Tool_환율조회`를 복제하고 이름을 `Tool_공휴일조회`로 변경
2. 트리거 노드의 Input Schema를 아래 한 필드로 교체

필드명	타입	Description
`country_code`	String	조회할 국가의 ISO 2자리 코드. 예: 일본 JP, 한국 KR, 미국 US, 베트남 VN

3. HTTP Request 노드의 URL을 아래로 교체

```
{{ 'https://date.nager.at/api/v3/NextPublicHolidays/' + $json.country_code }}
```

4. ▶ 버튼으로 HTTP Request까지 실행해 보면 OUTPUT에 **아이템이 여러 개**(10~16개) 표시됩니다. 이 상태로 Edit Fields를 거치면 문장도 여러 개가 만들어져 에이전트에게 목록이 통째로 넘어갑니다. 한 문장으로 합치기 위해 노드를 하나 끼워 넣습니다.
5. HTTP Request와 Edit Fields **사이**에 `Aggregate` 노드 추가

항목	값
Aggregate	`All Item Data (Into a Single List)`
Put Output in Field	`data`

Aggregate 노드는 들어온 아이템 전부를 `data`라는 하나의 배열 필드에 담아 **아이템 1개**로 만듭니다.

6. Edit Fields 노드의 `result` 값을 아래로 교체

```
{{ $('When Executed by Another Workflow').item.json.country_code + '의 다가오는 공휴일: '
+ $json.data.map(h => h.date + ' ' + h.localName).join(', ') }}
```

> 💡 `.map(h => ...)`은 배열의 각 항목을 "날짜 이름" 형태로 바꾸고, `.join(', ')`은 그것들을 쉼표로 이어 붙입니다. Expression 안에서 JavaScript 배열 함수를 그대로 쓸 수 있습니다.

7. 저장 후 5.2.2와 같은 방법으로 단독 테스트 (고정 데이터: `[{ "country_code": "JP" }]`) → `JP의 다가오는 공휴일: 2026-11-03 文化の日, 2026-11-23 勤労感謝の日, ...` 형태의 한 문장이 나오면 성공

5.5 실습 5-4. 출장 준비 비서 — Tool 세 개 연결

> 🎯 **목표**: Chapter 4의 업무 비서에 Tool 세 개를 연결해, 출장 준비에 필요한 정보를 한 번에 알려주는 에이전트를 완성한다.
> ⏱️ **소요 시간 15분**

이 실습이 해결하는 업무

해외 출장 전 날씨·환율·현지 공휴일을 각각 검색하던 일을 "도쿄 출장 준비해 줘" 한 마디로 끝냅니다.

5.5.1 에이전트에 연결하기

> ⚠️ **Tool 워크플로우는 반드시 Publish합니다.** 현재 n8n 2.x에서는 Agent가 Call n8n Workflow Tool로 호출하는 서브 워크플로우가 **Publish된 버전**으로 실행됩니다. 편집 화면의 채팅창으로 테스트할 때도 마찬가지이며, Publish하지 않으면 Tool 결과로 `Workflow is not active and cannot be executed.`가 돌아와 Agent가 "조회할 수 없다"고 답합니다. Tool 워크플로우를 수정한 뒤에도 **다시 Publish**해야 수정 내용이 반영됩니다.

1. `Tool_날씨조회`, `Tool_환율조회`, `Tool_공휴일조회`를 차례로 열어 각각 우측 상단 **Publish**를 클릭하고, 버튼이 **Published**로 바뀐 것을 확인
2. Chapter 4에서 만든 `실습4_업무비서`를 열고, **Duplicate**로 복제해 `실습5_출장준비비서`로 저장
3. AI Agent 노드의 `Tool` 커넥터에 `Call n8n Workflow Tool`을 **세 개** 추가하고 각각 아래와 같이 설정하고, **노드 이름을 반드시 아래 영문 이름으로 바꿉니다.**

> ⚠️ **Tool 노드 이름은 영문·숫자·밑줄(_)만 사용합니다.** Call n8n Workflow Tool은 노드 이름이 곧 Tool 이름이며, n8n은 그 외의 글자(한글, 공백)를 밑줄로 바꿉니다. 노드

> 이름을 `날씨 조회`, `환율 조회`, `공휴일 조회`로 두면 세 Tool의 이름이 모두 `_`로 같아져 Agent가 도구를 구분하지 못하고 모델 호출 단계에서 오류가 납니다.

노드 이름	Source for Workflow → Database	Description
`get_weather`	`Tool_날씨조회`	사용자가 특정 지역의 현재 날씨, 기온, 바람에 대해 물어볼 때 사용합니다. latitude와 longitude에는 해당 지역의 위도와 경도를, location_name에는 지역 이름을 전달하세요.
`get_exchange_rate`	`Tool_환율조회`	사용자가 환율이나 외화 환산을 물어볼 때 사용합니다. base_currency에는 기준 통화 코드(예: JPY), target_currency에는 환산할 통화 코드(보통 KRW)를 전달하세요.
`get_holidays`	`Tool_공휴일조회`	사용자가 특정 국가의 공휴일, 휴무일, 현지 휴일을 물어볼 때 사용합니다. country_code에는 ISO 2자리 국가 코드(예: JP, US)를 전달하세요.

3. 각 Tool의 `Workflow Inputs`에서 **Refresh** 클릭 → 필드가 나타나면 모두 **AI 버튼을 켜기**
4. System Message에 아래 두 줄을 추가

```
- 출장이나 해외 일정 준비를 요청받으면 날씨, 환율, 공휴일 도구를 모두 사용해 한 번에 정리해 주세요.
- 도구 결과에 없는 내용(항공편, 호텔 등)은 안내할 수 없다고 답하세요.
```

5.5.2 테스트

채팅창에서 먼저 Tool 하나씩 확인합니다.

```
"오늘 부산 날씨 어때?"
"지금 100엔이 몇 원이야?"
"한국 다음 공휴일 언제야?"
```

각 질문마다 **Logs 탭**에서 Tool이 호출되었는지, 입력값(`latitude`/`longitude`, `base_currency`, `country_code`)에 에이전트가 어떤 값을 채웠는지, Tool이 반환한 `result` 문장이 무엇인지 확인합니다. 에이전트가 좌표와 국가 코드를 스스로 만들어냈다는 점이 핵심입니다. 우리는 어디에도 알려주지 않았습니다.

이어서 세 Tool을 한 번에 쓰는 질문을 입력합니다.

```
"다음 주에 도쿄 출장 가는데 현지 날씨, 엔화 환율, 일본 공휴일 정리해 줘."
```

Logs 탭에 세 Tool이 차례로 호출되고, 결과를 합쳐 하나의 답변이 만들어지는 과정이 표시됩니다. 이것이 Chapter 1에서 이야기한 "실행 경로가 가변적"이라는 말의 실제 모습입니다. 질문에 따라 매번 다른 도구가, 다른 순서로 선택됩니다.

> 💡 환율 Tool은 유럽중앙은행 고시 기준이라 주말·공휴일에는 직전 영업일 값이 나옵니다. 답변의 날짜를 확인하는 습관이 필요합니다.

🔬 Description의 힘 확인해보기 (5분)

1. `get_weather` Tool의 Description을 `정보를 조회합니다.` 한 줄로 **일부러 모호하게** 바꿉니다
2. 다시 "오늘 부산 날씨 어때?"라고 물어봅니다
3. Logs 탭을 확인합니다 → Tool을 호출하지 않고 그냥 모른다고 답하거나, 엉뚱한 값을 채워 넣는 경우가 많습니다
4. Description을 원래대로 되돌리고 다시 테스트합니다

에이전트는 워크플로우 내부를 보지 않습니다. **Description만 읽고 판단합니다.** Tool이 호출되지 않는 문제의 대부분이 여기서 발생합니다.

트러블슈팅

증상	원인	해결
Tool을 호출하지 않고 텍스트로만 답변	Description이 모호함	Description을 구체적으로, 사용 시점을 명시
Workflow Inputs에 필드가 안 보임	서브 워크플로우 미저장 또는 Refresh 안 함	서브 워크플로우 저장 후 Refresh 클릭
값이 비어 있거나 이상하게 채워짐	AI 버튼이 꺼져 있음	각 필드의 AI 버튼 켜기
Logs의 Tool 결과가 `Workflow is not active and cannot be executed.`	서브 워크플로우를 Publish하지 않음	해당 Tool 워크플로우를 열어 Publish
서브 워크플로우를 고쳤는데 결과가 그대로임	수정 후 다시 Publish하지 않음 (Published, has changes 상태)	서브 워크플로우에서 다시 Publish
Tool 실행 중 오류	서브 워크플로우 자체의 문제	서브 워크플로우를 단독으로 실행해 확인
공휴일 결과가 여러 줄로 쪼개져 옴	Aggregate 노드 누락	HTTP Request와 Edit Fields 사이에 Aggregate

증상	원인	해결
		확인
환율 결과가 `undefined`	`target_currency` 코드 오타 또는 미지원 통화	3자리 대문자 코드 확인 (KRW, JPY, USD, EUR 등)

✅ 체크포인트 질문

- 공휴일 API처럼 아이템이 여러 개로 돌아올 때 하나로 합치는 노드는? → **Aggregate**
- 날씨·환율·공휴일이 모두 필요한 질문에서 에이전트는? → **필요한 Tool을 순서대로 호출한 뒤 결과를 조합**
- 1일차 Chapter 4의 `get_current_weather` 함수와 `Tool_날씨조회` 워크플로우의 공통점은? → **입력(도시/좌표) → API 호출 → 읽기 쉬운 결과 반환. 구현 방법만 다름**

🎉 완성 확인

"부산 날씨"에는 날씨 Tool을, "엔화 환율"에는 환율 Tool을, "일본 공휴일"에는 공휴일 Tool을 각각 알아서 호출하고, 출장 준비 질문에는 세 개를 모두 호출해 정리된 답변을 주면 실습 완료입니다. **Save**합니다.

📌 Chapter 5 요약

개념	핵심 내용
Workflow as Tool	서브 워크플로우(Execute Sub-workflow Trigger 시작)를 Call n8n Workflow Tool로 연결
Input Schema	에이전트가 채울 입력 필드를 정의. **Description에 예시를 넣으면 정확도가 오름**
AI 버튼	켜면 대화 맥락에 따라 모델이 값을 자동으로 채움
Tool Description	에이전트는 워크플로우 내부를 보지 않음. **Description만 보고 호출을 판단**
Aggregate	여러 아이템을 하나의 배열 필드로 합쳐 아이템 1개로 만듦
`$json.rates[변수]`	필드 이름이 입력값에 따라 달라질 때 대괄호로 접근
여러 Tool 조합	하나의 질문에 필요한 Tool들을 순서대로 호출해 결과를 조합
서브 워크플로우 테스트	에이전트에 붙이기 **전에** 반드시 단독 실행으로 검증
서브 워크플로우 Publish	Agent는 Publish된 버전만 호출. 수정 후에도 다시 Publish
실습 5-1~5-4	날씨·환율·공휴일 Tool 3개를 가진 출장 준비 비서 완성

다음 Chapter 예고: Chapter 6에서는 지금까지 배운 것을 모두 합쳐 실전 프로젝트를 만듭니다. 사내 규정 FAQ에 자동 응답하고, 답할 수 없는 질문은 담당자에게 넘기며, 이관 내역을 n8n Data Table에 기록하는 **사내 헬프데스크 Agent**입니다. 1일차 Chapter 6에서 코드로 만든 것과 같은 Agent를 노드로 만듭니다.